CSCI 499

Persona Messaging App

Maya Tabbah

Mushfiqur Rahman

James Chen

Savera Hayat

Write your message

Project Description

- Randomly connects users to chat with strangers
- No names or identifiable information shown by default
- Time-limited conversations before switching to a new user
- Users control how much personal information they share
- One-on-one direct conversations

Features

- **Full Anonymity:** No visible names, usernames, age, or personal details (usernames exist only for internal account management.)
- **Random Daily Matchmaking:** Users receive 3 random matches per day, each with a 20-minute timed chat.
- **Timed Interactions:** Short chats promote focused, intentional conversations; users can extend sessions using tokens.
- **Safety & Verification (TBD):** Identity verification required to prevent minors and AI bots from joining.
- **Planned Security Enhancements (TBD):** Encryption to be implemented in future updates for added privacy protection.

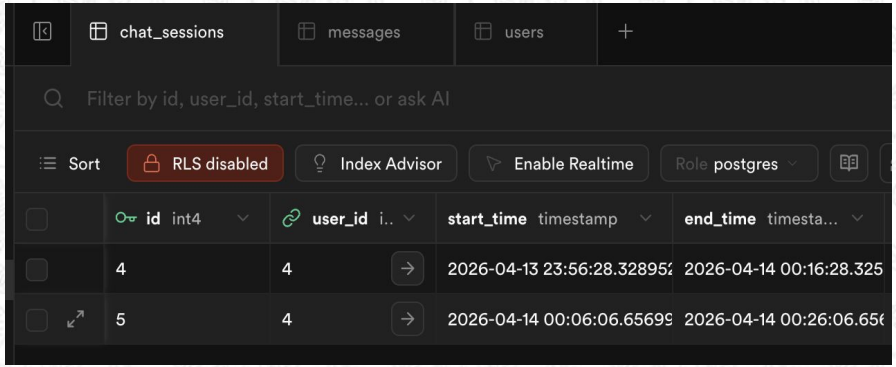
Real-Time Communication

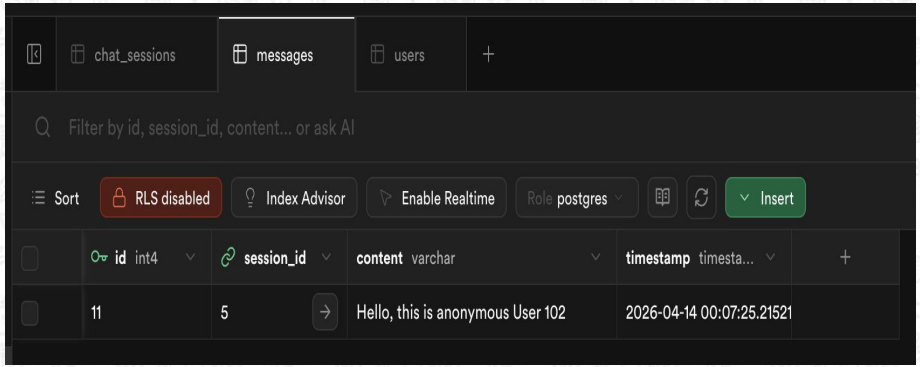
- **Infrastructure:** Moved the project from standard polling to WebSockets to enable the instantaneous communication required for a "live" chat experience.
- **Connection Management:** Developed a custom class to track active users and broadcast messages in real-time.

```python
@app.websocket("/ws/{session_id}/{client_id}")
async def websocket_endpoint(websocket: WebSocket, session_id: int, client_id: int):
    await manager.connect(websocket, session_id)
    db = SessionLocal()
    try:
        while True:
            data = await websocket.receive_text()

            # Save to Database
            new_msg = schemas.MessageCreate(content=data, session_id=session_id)
            crud.create_message(db, new_msg)

            # Broadcast to the room
            await manager.send_to_session(f"User {client_id}: {data}", session_id)

    except WebSocketDisconnect:
        manager.disconnect(websocket, session_id)
    finally:
        db.close()
```

Data Modeling & Cloud Persistence

- **Relational Schema**: Designed three core tables: Users, ChatSessions, and Messages (with Foreign Key relationships to ensure data integrity)
- **The ORM Layer**: Utilized SQLAlchemy to bridge Python objects with our SQL database, ensuring that all data interactions remain type-safe and efficient.
- **Cloud Integration (Challenge)**: Had to migrate our local SQLite development environment to a live Supabase instance, providing a centralized cloud home for our project data.





Matchmaking Algorithm

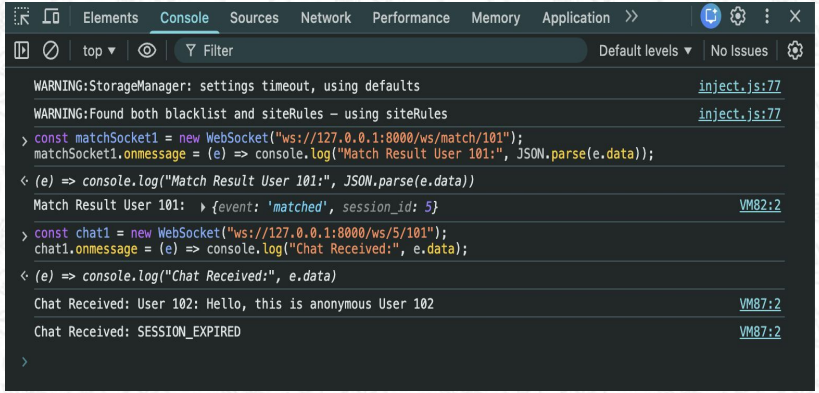
- **Queue Logic**: Implemented a waiting room queue. When a user joins, the system checks the queue size. If it hits 2, a new session_id is automatically generated in SQL to pair them.
- **Removing Users (Challenge)**: Had to also ensure that if a user closed their browser while waiting, they were immediately removed from the queue to prevent pairing an active user with an offline one.

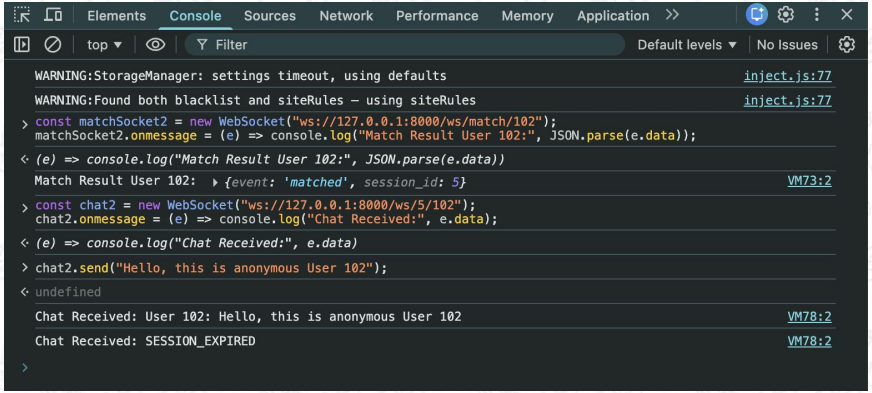
```python
class ConnectionManager:
    def __init__(self):
        # Maps session_id -> list of WebSockets
        self.active_connections: Dict[int, List[WebSocket]] = {}
        # List of WebSockets waiting for a match
        self.waiting_room: List[Dict] = []

    async def add_to_waiting_room(self, websocket: WebSocket, user_id: int):
        await websocket.accept()
        self.waiting_room.append({"ws": websocket, "user_id": user_id})
        print(f"DEBUG: User {user_id} added to waiting room. Queue size: {len(self.waiting_room)}")

    async def try_match(self, db: SessionLocal):
        if len(self.waiting_room) >= 2:
            # Pop the first two users
            user1 = self.waiting_room.pop(0)
            user2 = self.waiting_room.pop(0)
```

```
(venv) james@Mac backend % uvicorn app.main:app --reload
INFO:     Started server process [20968]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:59053 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:59053 - "GET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:59053 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:59053 - "GET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:59071 - "WebSocket /ws/match/101" [accepted]
DEBUG: User 4 added to waiting room. Queue size: 1
INFO:     connection open
INFO:     127.0.0.1:59104 - "WebSocket /ws/match/102" [accepted]
DEBUG: User 5 added to waiting room. Queue size: 2
INFO:     connection open
INFO:     127.0.0.1:59302 - "WebSocket /ws/5/101" [accepted]
DEBUG: User joined Session 5. Total in room: 1
INFO:     connection open
INFO:     127.0.0.1:59412 - "WebSocket /ws/5/102" [accepted]
DEBUG: User joined Session 5. Total in room: 2
INFO:     connection open
DEBUG: Broadcasting to 2 users in Session 5
DEBUG: Broadcasting to 2 users in Session 5
DEBUG: Session 5 has expired.
```

Automated Session Expiry

- **Timed Persistence (Challenge):** I integrated an automated timer using asyncio.sleep to monitor the 20-minute chat window.
- **Automated Connection Close:** Upon timer completion, the server broadcasts a SESSION_EXPIRED event and closes the connection.
- **SQL Updates:** I implemented logic to mark the session as is_expired = True in Supabase to prevent further messages from being saved to the session (still fixing).





Lessons Learned & Future Outlook

- **Technical:** I learned that WebSockets require much more rigorous state management than standard REST APIs, especially when handling unexpected disconnects.
- **Architecture:** Transitioning from a managed service like Supabase to a custom FastAPI setup was difficult but necessary.
- **Refining the Matcher:** My next step is to improve the matching algorithm.
- **Deployment:** Moving the current local Uvicorn server to a production-ready environment.
- **Integration:** Bridging the remaining gap between the React frontend components and the established WebSocket routes.

Persona Messaging App

11:25 AM

Thank You!

https://github.com/maya-tabbah/Capstone_Group_6